

Smart Contract Audit Report

Wallitelli (WALLI)

BEP-20 Token, BNB Smart Chain

A full review of the deployed WallitelliToken contract: source against deployed bytecode, the complete callable surface, the inherited OpenZeppelin ERC-20 behaviour, and the centralisation profile of the launch allocation. The audit was performed in May 2026 and every factual statement in this edition was re-verified against the live chain on 22 September 2026.

OVERALL RISK RATING **LOW**

PREPARED BY SOKEN LTD, Smart Contract Security	REPORT ID WLT-2026-05-001, edition 22 September 2026
AUDIT PERIOD May 2026, report issued 22 May 2026	FIGURES RE-VERIFIED 22 September 2026, block 123,343,421
NETWORK BNB Smart Chain (chainId 56)	FINDINGS 0 Critical, 0 High, 0 Medium, 1 Low, 4 Informational
TOKEN CONTRACT 0xc6A031fd206C72c614dB391590F48a5Ed9B2f9dC	COMPILER solc 0.8.28, optimizer enabled, 800 runs, EVM paris

SOKEN LTD. Security review of on-chain source code. Not financial, investment or legal advice. See the disclaimer in section 08.

01 Executive summary

WallitelliToken is a fixed-supply BEP-20 token on BNB Smart Chain. The contract is 12 source lines long. It inherits the OpenZeppelin 5.4.0 ERC20 base without overriding a single function, adds one public constant, and mints the entire supply once in its constructor. There is no owner, no pause switch, no blacklist, no mint or burn path after deployment, no fee on transfer and no upgrade mechanism. We re-confirmed each of those absences by calling the selectors against the live contract: every one reverts, and the contract has no fallback function.

The audit found no defect in the contract. The single Low finding is a property of the launch model rather than of the code: the whole supply is created in one transaction and handed to the deploying account, so holders depend on off-chain commitments for how it is distributed. The four Informational findings are hygiene and documentation items.

- **The deployed code is the audited code.** We compiled the explorer-verified source locally with the compiler settings recorded in the contract metadata; the resulting runtime bytecode is byte-for-byte identical to the code returned by the chain (1,888 bytes). Sourcify independently reports a full creation and runtime match for this address.
- **Supply is fixed and cannot grow.** `totalSupply()` returns 1,000,000,000 WALLI today, the same figure minted at deployment on 7 May 2026. No code path can mint or burn.
- **No privileged role exists.** No account, including the deployer, can pause transfers, seize a balance, change metadata or replace the logic. The deployer's power is bounded by the tokens it holds.
- **Re-verification suite.** A fresh suite of 48 cases (43 unit cases and 5 fuzz properties at 512 runs each) was written against the verified source and executed on 22 September 2026: 48 of 48 passing. Slither and Aderyn were re-run on the same source and produced no High or Medium item.

0

CRITICAL, HIGH AND MEDIUM FINDINGS

1 + 4

LOW AND INFORMATIONAL FINDINGS

12

SOURCE LINES UNDER AUDIT

48 / 48

RE-VERIFICATION TESTS PASSING

byte-exact

SOURCE AGAINST DEPLOYED BYTECODE

1e27

BASE UNITS, SUPPLY UNCHANGED

02 Scope, method and re-verification

AREA	WHAT WE DID
Source under audit	Single file <code>WallitelliToken.sol</code> , 12 lines, 8 non-blank, plus the inherited OpenZeppelin 5.4.0 files <code>ERC20.sol</code> , <code>IERC20.sol</code> , <code>IERC20Metadata.sol</code> , <code>draft-IERC6093.sol</code> and <code>Context.sol</code> . Read line by line and diffed against the published OpenZeppelin 5.4.0 base.
Deployed code identity	Recompiled the verified source with solc 0.8.28, optimizer enabled, 800 runs, EVM target <code>paris</code> , metadata hash <code>none</code> . The runtime bytecode matched <code>eth_getCode</code> exactly. Sourcify: <code>creationMatch=match</code> , <code>runtimeMatch=match</code> , <code>verifiedAt 2026-06-02T02:27:40Z</code> .
Live surface probe	Called every administrative selector a token of this class might expose against the live contract: <code>owner</code> , <code>transferOwnership</code> , <code>renounceOwnership</code> , <code>pause</code> , <code>paused</code> , <code>blacklist</code> , <code>mint</code> , <code>burn</code> , <code>burnFrom</code> , <code>upgradeTo</code> , <code>implementation</code> , <code>permit</code> , <code>DOMAIN_SEPARATOR</code> , <code>nonces</code> . All revert, and a plain value call reverts as well.

Behavioural testing	48 Foundry cases against the verified source: metadata and supply, transfer and allowance semantics including infinite allowance, EIP-6093 revert shapes, the absence of every administrative selector, reentrancy exposure of the recipient, gas boundedness, and 5 fuzz properties over supply and balance conservation.
Static analysis	Slither: 5 informational (pragma drift in OpenZeppelin dependencies), 0 low, 0 medium, 0 high. Aderyn: 0 high, 2 low (large numeric literal, unspecific pragma). Both re-run on 22 September 2026 against the same verified source.
Threat model	Enumerated the standard ERC-20 attack classes (supply manipulation, admin escalation, allowance race, reentrancy through the recipient, fee or rebase hooks, upgrade hijack, oracle and flash-loan dependence, selfdestruct) and checked in each case whether the surface exists at all.

Out of scope

Off-chain key management of the deploying account, exchange and listing integrations, liquidity pool configuration, the front end, and the marketing or tokenomics documents. This report covers the on-chain contract.

03 Contract surface and trust model

What the contract exposes, and what it deliberately does not



The left column is the inheritance chain, the middle column is the callable surface exercised by the test suite, and the right column lists the selectors that are absent. Each absence was confirmed by calling the selector against the live contract at block 123,343,421; the call reverts because no such function exists and there is no fallback.

ACTOR	POWER ON-CHAIN
Deploying account 0x35A52b...a67CC5	Received the entire supply in the deployment transaction. After construction it has exactly the rights of any other holder: move its own balance, grant allowances. It cannot mint, burn, pause, freeze, or reach another holder's balance.
Token holders	Transfer their own balance, grant and revoke allowances, read public state.
Anyone	Read metadata and balances. No permissionless state change beyond ordinary ERC-20 calls.
Administrator	Does not exist. There is no role, no owner slot and no gated function in the contract.

Inherited behaviour worth stating

- Failure reverts with EIP-6093 custom errors rather than returning false. Integrators must not rely on a boolean return to detect failure.
- An allowance equal to `type(uint256).max` is treated as infinite and is not decremented on `transferFrom`.
- A transfer does not call the recipient, so a recipient contract cannot re-enter the token during a transfer.
- The classic ERC-20 approval race is inherent to the standard and applies here as it does to any ERC-20.

04 Findings

ID	FINDING	SEVERITY	STATUS
W-01	Entire initial supply minted to a single deploying account	■ LOW	■ ACKNOWLEDGED
W-02	Floating compiler pragma in the source file	■ INFORMATIONAL	■ ACKNOWLEDGED
W-03	No EIP-2612 permit extension	■ INFORMATIONAL	■ ACKNOWLEDGED
W-04	Source licence declared as UNLICENSED	■ INFORMATIONAL	■ ACKNOWLEDGED
W-05	Immutability and the absence of an administrator are not stated on-chain	■ INFORMATIONAL	■ ACKNOWLEDGED

Severity distribution: 0 Critical, 0 High, 0 Medium, 1 Low, 4 Informational. No finding required a code change for the contract to be safe to use within its declared fixed-supply, no-administrator design.

W-01 Entire initial supply minted to a single deploying account

■ LOW

Location. `WallitelliToken.sol`:9-11, the constructor and its single `_mint` call.

The constructor mints 1,000,000,000 WALLI (1e27 base units) to `msg.sender`, which in this deployment was the externally owned account `0x35A52b...a67CC5` in transaction `0xc120a848...968fd707` at block 96,810,456, 2026-05-07 01:43 UTC. We confirmed on an archive node that the deploying account held the full supply immediately after construction, and that the account holds no contract code, so it is a plain key.

No vesting contract, treasury multi-signature wallet or distribution schedule takes part in construction. Holders therefore depend on off-chain commitments for how the supply is released. The contract itself gives the deployer no power over other holders: with no mint, burn, freeze or clawback function, the risk is concentration of the float, not seizure. The same pattern is used by most token launches, which is why this is rated Low rather than higher, but it is the one material centralisation property a prospective holder should understand.

Current position. At block 123,343,421 (2026-09-22 08:45 UTC) the deploying account holds 0 WALLI, so the initial allocation has been moved on since the audit. Tracing the recipients requires a full historical log scan, which the free public endpoints available on the re-verification date would not serve, so this edition makes no statement about where the supply sits today. A holder can read the current distribution from any BNB Smart Chain explorer.

Recommendation. Hold the launch allocation in a multi-signature wallet rather than a single key, move team, treasury and liquidity portions into audited vesting contracts with publicly verifiable schedules, and publish the distribution plan and the addresses that hold each portion so that the on-chain picture can be checked against it.

Status. Acknowledged. No change is owed in the contract; the remedy is in the distribution model.

W-02 Floating compiler pragma

■ INFORMATIONAL

Location. WallitelliToken.sol:2, pragma solidity ^0.8.28;.

The pragma accepts any 0.8.x release from 0.8.28 up to but excluding 0.9.0. The deployed bytecode was produced by one specific compiler, solc 0.8.28+commit.7893614a, so this has no effect on the live contract. It matters for reproducibility: anyone who recompiles the source later, for an indexer, an integration repository or a verification harness, may get different bytecode and a different known-bug profile. Both static analysers flag the same item.

Recommendation. Pin the pragma to pragma solidity 0.8.28; and record the compiler version, optimizer setting and EVM target in the repository so a rebuild is deterministic.

Status. Acknowledged. No behavioural impact on the deployed contract.

W-03 No EIP-2612 permit extension

■ INFORMATIONAL

Location. Contract-wide feature gap. Confirmed on-chain: DOMAIN_SEPARATOR() and nonces(address) revert.

Without permit, a user needs two transactions for any interaction where a contract pulls tokens: approve, then spend. Some routers and aggregators fall back to a less efficient path for tokens that do not support permit. The absence is a legitimate choice that keeps the signature surface minimal; it is recorded here so integrators do not assume support.

Recommendation. If gasless approvals are wanted later, they require a new token contract with the OpenZeppelin ERC20Permit extension, since this contract cannot be upgraded. If permit is intentionally out of scope, state that in the project documentation.

Status. Acknowledged. Feature request, not a defect.

W-04 Source licence declared as UNLICENSED

■ INFORMATIONAL

Location. WallitelliToken.sol:1, // SPDX-License-Identifier: UNLICENSED.

The identifier is valid and conveys the most restrictive posture: the code is publicly readable on the explorer but remains proprietary, so third parties who fork it or build derivative tooling around it have no licence to do so. For a token intended to be integrated widely this is usually not the intent.

Recommendation. If open re-use is intended, publish a new generation under a permissive identifier such as MIT, since the SPDX line in a deployed contract cannot be changed. Otherwise keep UNLICENSED and publish the licensing terms next to the token documentation.

Status. Acknowledged. A project decision rather than a security issue.

■ INFORMATIONAL

Location. Contract-wide documentation gap. The only self-describing value is `VERSION = "Wallitelli-v1"`.

The contract has no upgrade path and no privileged function, which is a positive security property, but nothing in the source or the ABI says so. An integrator reading the explorer page cannot easily distinguish "no administrative mechanism by design" from "an administrative mechanism I have not found yet", and that ambiguity costs the project credit it has earned.

Recommendation. State the fixed supply, the absence of an administrator and the absence of an upgrade path in the source comments of any future generation, in the project README and tokenomics document, and in the explorer contract page description, so the claim can be checked against the verified source.

Status. Acknowledged. Documentation only.

05 Verification evidence

Deployment state, read on 2026-09-22 08:45 UTC

ITEM	VALUE READ FROM THE CHAIN
Contract	0xc6A031fd206C72c614dB391590F48a5Ed9B2f9dC
Network	BNB Smart Chain, chainId 56, block 123,343,421
name / symbol / decimals	Wallitelli / WALLI / 18
totalSupply	10000000000000000000000000000 base units, 1,000,000,000 WALLI
VERSION	Wallitelli-v1
Deployment	block 96,810,456, 2026-05-07 01:43 UTC, transaction 0xc120a848...968fd707
Deploying account	0x35A52bc794e8Fa5402f9919F4fc756E464a67CC5 (no contract code, balance 0 WALLI today)
Runtime bytecode	1,888 bytes, identical to the local build of the verified source
Explorer verification	creationMatch=match, runtimeMatch=match, verifiedAt 2026-06-02T02:27:40Z
Administrative selectors	owner, transferOwnership, renounceOwnership, pause, paused, blacklist, mint, burn, burnFrom, upgradeTo, implementation, permit, DOMAIN_SEPARATOR, nonces: all revert

Test card, executed 22 September 2026

GROUP	CASES	RESULT
Metadata, supply and transfer semantics	16	■ CONFIRMED
Allowance semantics, including infinite allowance and EIP-6093 error shapes	11	■ CONFIRMED
Absence of administrative and upgrade selectors, no ether accepted, garbage calldata	9	■ CONFIRMED
Attack scenarios: recipient reentrancy, unauthorised movement, supply overflow, metadata spoofing, gas bound	7	■ CONFIRMED

Fuzz properties at 512 runs each: supply constant, balance conservation, unauthorised transfer always reverts, allowance fidelity, no mint path through arbitrary calldata	5	■ CONFIRMED
Total	48	48 passing, 0 failing

Static analysis, re-run 22 September 2026

TOOL	RESULT
Slither	5 informational (pragma drift in OpenZeppelin dependencies), 0 low, 0 medium, 0 high. Every result sits in the compiler-version detector class and concerns the pragma ranges declared by the OpenZeppelin dependency files, not the audited logic.
Aderyn	0 high, 2 low (large numeric literal, unspecific pragma). The large-literal item is a style note on <code>1_000_000_000 * 10 ** decimals()</code> ; the pragma item is finding W-02.

Changes in this edition. The May 2026 report described the deployment as compiled for the prague EVM target with PUSH0 in the bytecode. That was wrong: the metadata and the Sourcify compilation record show the paris target, and the deployed runtime code contains no PUSH0 opcode. The related static-analysis advisory has been dropped, the optimizer setting has been added, and the test card now cites the suite executed on the re-verification date rather than the original run. The full claim-by-claim record is kept with this report.

06 Conclusion

WallitelliToken is as small as a token contract can reasonably be, and that is its main security property. All behaviour comes from the unmodified OpenZeppelin 5.4.0 ERC20 base; the project-specific code is a one-shot mint and a version string. There is no administrative surface to abuse, no upgrade path to hijack and no supply mechanism to exploit, and the deployed bytecode provably corresponds to the published source.

The rating is LOW RISK with a security score of 96 out of 100 on our internal scale. The deduction reflects the launch concentration in W-01 and the hygiene items, not any defect in the code.

- Move the launch allocation into a multi-signature wallet and vesting contracts, and publish the addresses (W-01).
- Pin the compiler pragma and record the build settings for reproducibility (W-02).
- Publish a distribution and immutability statement linked from the explorer contract page (W-01, W-05).
- Decide the licensing posture deliberately and state it publicly (W-04).

No code change is required for the contract to be used in production within the bounds of its declared design. Any future feature, including permit, requires a new contract, because this one cannot be upgraded.

07 Appendix: full source under audit

The complete project-specific source, as verified against the deployed bytecode. Everything else in the contract comes from the unmodified OpenZeppelin 5.4.0 ERC20 base.

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.28;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract WallitelliToken is ERC20 {
    string public constant VERSION = "Wallitelli-v1";

    constructor() ERC20("Wallitelli", "WALLI") {
        _mint(msg.sender, 1_000_000_000 * 10 ** decimals());
    }
}
```

08 Disclaimer

This report was prepared by SOKEN LTD (Companies House 15253906) and covers the contract source and deployed bytecode described in section 02 at the addresses and blocks stated. It is a point-in-time technical review, not a guarantee that the code is free of defects, and it is not financial, investment or legal advice. Findings and figures were re-verified on 22 September 2026 at block 123,343,421; on-chain values change as the chain moves, so re-read any figure at the linked address or transaction before relying on it. Off-chain conduct, key management, distribution decisions and third-party integrations are outside the scope of this review.